

Chuyên đề: Quy Hoạch Động Chữ Số (Digit DP)

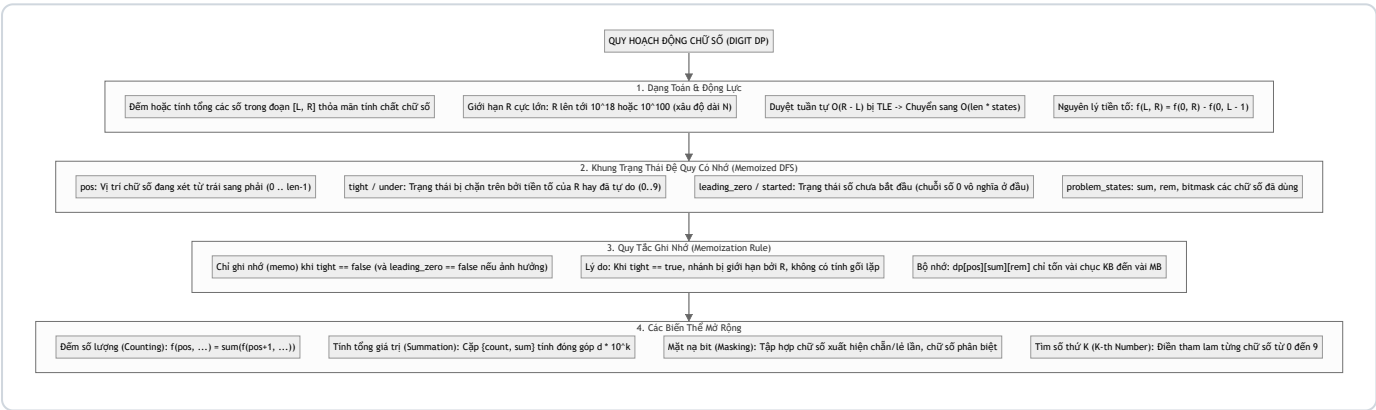
(Digit Dynamic Programming: Counting, Summation, Digit Masking, Modulo Constraints & K-th Number on $[L, R]$)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Đặt Vấn Đề: Tại Sao Cần Digit DP?

1.1. Giới hạn của phương pháp duyệt thô (Brute-force)

Xét bài toán kinh điển: Đếm số lượng số nguyên X trong đoạn $[L, R]$ sao cho tổng các chữ số của X chia hết cho K ($1 \leq L \leq R \leq 10^{18}$, $K \leq 100$).

Nếu ta duyệt từng số X từ L đến R :

- Với $R \leq 10^6$: Thuật toán chạy trong khoảng vài phần mười giây.
- Với $R \leq 10^{18}$: Cần thực hiện 10^{18} phép tính $\implies$ Mất hàng thế kỷ để chạy xong trên siêu máy tính!

1.2. Bản chất của Quy hoạch động chữ số

Nhận xét quan trọng:

- Số 10^{18} tuy có giá trị rất lớn, nhưng trong hệ thập phân nó chỉ có **đúng 19 chữ số** ($\approx \log_{10} R$).
- Thay vì duyệt qua từng giá trị, ta **xây dựng từng chữ số** của số từ trái sang phải (từ hàng cao nhất xuống hàng đơn vị).
- Tại mỗi vị trí chữ số, có tối đa 10 lựa chọn (từ 0 đến 9).

4. Các số có cùng tiền tố tương đương (ví dụ có cùng độ dài còn lại, cùng tổng chữ số dư, không bị ràng buộc bởi cận trên) sẽ dẫn đến **cùng số lượng cách hoàn tất số**. Đây chính là tính chất **gối nhau của các bài toán con (Overlapping Subproblems)**!

Độ phức tạp chuyển từ $O(R - L)$ thành:

$$O(\log_{10} R \times \text{Số trạng thái con}) \approx 19 \times 100 \times 10 \approx 1.9 \times 10^4 \text{ phép tính}$$

Chương trình chạy trong **chưa đầy 0.001 giây**!

2. Nguyên Lý Tiền Tố (Prefix Reduction Principle)

Hầu hết các bài toán Digit DP yêu cầu giải trên đoạn đóng $[L, R]$. Ta luôn đưa về hiệu của hai bài toán trên đoạn $[0, X]$:

$$f(L, R) = f(0, R) - f(0, L - 1)$$

(Lưu ý: Nếu $L = 0$, kết quả là $f(0, R)$).

Lợi ích của nguyên lý này:

- Ta chỉ cần viết duy nhất **một hàm giải cho bài toán** $[0, N]$ với cận trên N cố định.
- Không phải đồng thời kiểm soát cả cận dưới L và cận trên R trong cùng một hàm đệ quy.

3. Khung Cấu Trúc Đệ Quy Có Nhớ (Standard Digit DP Framework)

Chuyển số N thành chuỗi các chữ số: $S = d_0 d_1 d_2 \dots d_{m-1}$ (với d_0 là chữ số có trọng số lớn nhất).

3.1. Các tham số trạng thái cốt lõi

```
long long dfs(int pos, bool tight, bool leading_zero, int state1, int state2, ...);
```

Tham số	Ý nghĩa	Miền giá trị
<code>pos</code>	Vị trí chữ số hiện tại đang cần điền (từ trái sang phải)	$0 \leq pos < m$
<code>tight</code>	Cờ báo hiệu có bị "bó chặt" bởi tiền tố của số N hay không	<code>true</code> / <code>false</code>
<code>leading_zero</code>	Cờ báo hiệu các chữ số trước đó toàn là số 0 vô nghĩa ở đầu	<code>true</code> / <code>false</code>
<code>state</code>	Các thuộc tính đề bài yêu cầu (tổng chữ số, số dư modulo K , bitmask...)	Phụ thuộc đề bài

3.2. Ý nghĩa của cờ `tight` (Ràng buộc biên trên)

Giả sử số $N = 3524$.

- Tại vị trí đầu tiên ($pos = 0$): Chữ số d_0 của N là 3.
- Nếu ta chọn điền chữ số $c \in \{0, 1, 2\}$: Các chữ số ở các vị trí tiếp theo ($pos = 1, 2, 3$) có thể thoải mái

chọn từ 0 đến 9 mà **chắc chắn không bao giờ vượt quá 3524**. Lúc này `tight` chuyển từ `true` thành `false` (trở nên tự do).

- Nếu ta chọn điền đúng chữ số $c = 3$: Ở vị trí tiếp theo ($pos = 1$), ta chỉ được phép chọn chữ số từ 0 đến 5 (chữ số tiếp theo của N). Lúc này `tight` **vẫn tiếp tục là** `true`.

Quy tắc cập nhật cờ `tight`:

$$\text{new_tight} = \text{tight} \wedge (c == \text{limit})$$

trong đó:

$$\text{limit} = \begin{cases} d_{pos}, & \text{nếu } \text{tight} == \text{true} \\ 9, & \text{nếu } \text{tight} == \text{false} \end{cases}$$

3.3. Ý nghĩa của cờ `leading_zero` (Số 0 vô nghĩa ở đầu)

Trong toán học, số 7 được biểu diễn với độ dài 4 chữ số là 0007.

- Nếu bài toán yêu cầu: "Không có hai chữ số liền kề nào giống nhau" (ví dụ: CSES 2220 - Counting Numbers).
- Số 0007 thực chất là số 7, chỉ có 1 chữ số. Cặp (0, 0) ở đầu **không vi phạm** điều kiện bài toán!
- Nếu không có cờ `leading_zero`, thuật toán sẽ coi cặp (0, 0) là hai chữ số giống nhau và loại bỏ oan số 7.
- Khi `leading_zero == true` và ta chọn tiếp tục điền chữ số 0: Số vẫn chưa thực sự bắt đầu, `leading_zero` tiếp tục là `true`.
- Ngay khi ta điền một chữ số $c > 0$: Số chính thức bắt đầu, `leading_zero` trở thành `false`.

4. Quy Tắc Vàng Về Ghi Nhớ (Memoization Rule)

[!CRITICAL]

Chỉ được phép đọc và lưu giá trị vào mảng nhớ `dp` khi `tight == false` (và `leading_zero == false` nếu trạng thái bắt đầu ảnh hưởng đến bài toán)!

Tại sao không lưu khi `tight == true`?

- Khi `tight == true`, kết quả trả về phụ thuộc chặt chẽ vào các chữ số cụ thể của N đứng sau vị trí đó. Số lượng trạng thái `tight == true` trên cây đệ quy chỉ là **duy nhất một nhánh** đi dọc theo các chữ số của N (tối đa ≈ 19 nút).
- Việc ghi nhớ chỉ thực sự có ý nghĩa cho các nhánh `tight == false` (nơi không gian tìm kiếm là toàn bộ $[0, 99 \dots 9]$, được tái sử dụng hàng nghìn lần giữa các nhánh khác nhau).
- Do đó, mảng `dp` **không cần** chiều `[tight]` (hoặc nếu có lưu chiều `[tight]`, kích thước tăng gấp đôi nhưng không gây sai số). Tối ưu nhất là:

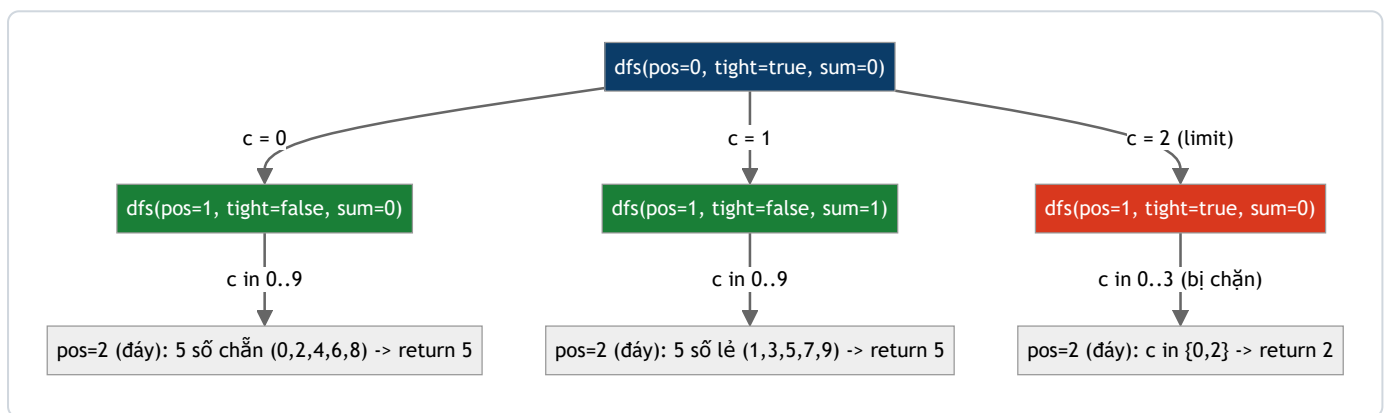
```

if (!tight && !leading_zero && dp[pos][state] != -1) {
    return dp[pos][state];
}
// ... tính toán res ...
if (!tight && !leading_zero) {
    dp[pos][state] = res;
}
return res;

```

5. Trace Chi Tiết Bước Chuyển Trạng Thái (Step-by-Step Trace)

Xét bài toán: Đếm số lượng số trong đoạn $[1, 23]$ có tổng các chữ số là số chẵn ($sum \pmod 2 == 0$).
Chuỗi chữ số của $N = 23$ là $S = ['2', '3']$, độ dài $m = 2$.



Bảng phân tích kết quả:

- Nhánh $c = 0$ ($pos = 0$): Tạo ra các số $00, 01, \dots, 09$. Các số có tổng chẵn là $\{0, 2, 4, 6, 8\} \implies 5$ số. (Được lưu vào `dp[1][0] = 5`).
- Nhánh $c = 1$ ($pos = 0$): Tạo ra các số $10, 11, \dots, 19$. Các số có tổng chẵn là $\{11, 13, 15, 17, 19\} \implies 5$ số. (Được lưu vào `dp[1][1] = 5`).
- Nhánh $c = 2$ ($pos = 0, tight=true$): Vì bị chặn bởi chữ số 3, chỉ được chọn $c \in \{0, 1, 2, 3\}$. Các số tạo thành là $\{20, 21, 22, 23\}$. Số có tổng chẵn là $\{20, 22\} \implies 2$ số.
- Tổng cộng trong đoạn $[0, 23]$: $5 + 5 + 2 = 12$ số (gồm cả số 0).
- Loại trừ số 0 (vì đề hỏi $[1, 23]$): Đáp án là $12 - 1 = 11$ số.

6. Các Mô Hình Biến Thể Thường Gặp

6.1. Biến thể 1: Ràng buộc Modulo / Chia hết

- Đặc trưng:** Tìm số chia hết cho K , hoặc tổng chữ số chia hết cho K .
- Trạng thái:** `rem = (rem * 10 + c) % K` hoặc `sum = (sum + c) % K`.
- Đáy đệ quy** ($pos == m$): Trả về 1 nếu `rem == 0`, ngược lại trả về 0.

6.2. Biến thể 2: Tính tổng giá trị (Sum of Numbers)

- **Đặc trưng:** Thay vì đếm số lượng, đề bài yêu cầu tính **tổng giá trị** của tất cả các số thỏa mãn điều kiện trong đoạn $[L, R]$ (ví dụ: *Codeforces 1073E - Segment Sum*).
- **Kỹ thuật đóng góp giá trị (Contribution Technique):**

Mỗi hàm đệ quy trả về một cặp giá trị: `struct Node { long long count; long long sum; }; .`

Nếu ở vị trí pos , ta điền chữ số c , thì chữ số c này đóng góp vào giá trị của mỗi số hợp lệ ở nhánh con một lượng là:

$$c \times 10^{\text{len}-1-pos} \pmod{MOD}$$

Tổng giá trị thu được từ nhánh con là:

$$\text{total_sum} = \sum (\text{child.sum} + \text{child.count} \times c \times 10^{\text{len}-1-pos}) \pmod{MOD}$$

6.3. Biến thể 3: Mặt nạ Bitmask chữ số (Digit Masking)

- **Đặc trưng:** Đếm số có không quá K chữ số khác nhau, hoặc các chữ số xuất hiện với tính chẵn lẻ nhất định.
- **Trạng thái:** Dùng số nguyên `mask` (10 bits từ 0 đến 9).
- Khi điền chữ số c : `new_mask = mask | (1 << c) .`
- Nếu xét chẵn lẻ: `new_mask = mask ^ (1 << c) .`
- **Số trạng thái:** $2^{10} = 1024$, hoàn toàn khả thi khi kết hợp với $pos \leq 19$.

6.4. Biến thể 4: Tìm số thứ K thỏa mãn điều kiện (K-th Valid Number)

- **Phương pháp 1 (Chặt nhị phân):** Nếu tính chất thỏa mãn tính đơn điệu của hàm đếm $\text{count}(X)$, ta có thể chặt nhị phân giá trị $X \in [1, 10^{18}]$ sao cho $f(1, X) = K$. Mỗi bước kiểm tra gọi hàm Digit DP trong $O(\log_{10} X \cdot \text{states})$. Tổng thời gian: $O(\log_2(10^{18}) \cdot \log_{10}(10^{18}) \cdot \text{states}) \approx 60 \times 19 \times \text{states}$.
- **Phương pháp 2 (Điền tham lam từng chữ số - Digit DP Trial):** Duyệt từng vị trí từ trái sang phải, thử điền từng chữ số $c \in [0, 9]$. Gọi hàm Digit DP đếm xem nếu điền c thì có bao nhiêu số hợp lệ. Nếu số lượng $\geq K$, chữ số tại vị trí này chắc chắn là c ; nếu $< K$, ta trừ bớt K đi lượng đó và thử chữ số tiếp theo $c + 1$.

7. Cạm Bẫy Phòng Thi & Kinh Nghiệm Tối Ưu

1. **Không xóa sạch mảng `dp` giữa các test cases:**
2. Nếu hàm Digit DP độc lập với cận trên N (trạng thái chỉ phụ thuộc vào pos, sum, rem), mảng `dp` **có thể dùng chung** giữa các test cases mà không cần `memset` lại! Điều này giúp giải quyết các bài toán có hàng ngàn truy vấn ($T \leq 10^5$) trong $O(1)$ mỗi test.
3. Tuy nhiên, nếu trạng thái phụ thuộc vào tham số của test (như K thay đổi theo từng test), **bắt buộc** phải `memset(dp, -1, sizeof(dp))` cho mỗi test case.

4. Xử lý cận dưới $L = 0$ và $L = 1$:

5. Thao tác $f(0, R) - f(0, L - 1)$ sẽ gặp lỗi nếu $L = 0$ vì $L - 1 = -1$.

6. Luôn viết hàm phụ trợ:

```
cpp long long query(long long L, long long R) { long long ans = solve(R); if (L > 0) ans -= solve(L - 1); return ans; }
```

7. Cờ `leading_zero` bị bỏ quên:

8. Trong các bài toán có tích các chữ số, số các chữ số 0, hoặc chữ số liền kề: Các số 0 đứng trước số có nghĩa (như 0005) không được tính vào tích hoặc tính vào số lượng chữ số 0. Luôn truyền cờ `leading_zero` vào hàm đệ quy.

9. Tràn số với modulo:

10. Khi tính hiệu $f(0, R) - f(0, L - 1) \pmod{MOD}$, kết quả có thể âm:

```
cpp ans = (ans % MOD + MOD) % MOD;
```

8. Tài liệu Tham khảo & Đọc thêm

1. **CP-Algorithms:**

2. [Digit DP Tutorial & Applications](#) — Hướng dẫn phương pháp đệ quy có nhớ cho bài toán chữ số.

3. **Codeforces Educational Resources:**

4. [Digit DP Comprehensive Blog](#) — Tổng hợp các mẫu bài toán Digit DP từ cơ bản đến nâng cao.

5. **GeeksforGeeks:**

6. [Digit DP Introduction](#) — Phân tích chi tiết trạng thái `tight` và cách chuyển đổi.

7. **VNOI Wiki:**

8. [Quy hoạch động chữ số](#) — Tài liệu tiếng Việt chuẩn mực cho sinh viên Olympic Tin học Việt Nam.

9. **CSES Problemset:**

10. [CSES 2220 - Counting Numbers](#) — Bài tập kinh điển kiểm tra kỹ năng xử lý `leading_zeros`.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.